

Yep — this is an important piece of the **HTTP → HTTPS → authentication/cookies** picture. 🗝️

What is TLS?

TLS = Transport Layer Security.

TLS is the security protocol that **protects HTTP communication while it travels between the browser and server.**

So:

HTTPS = HTTP + TLS

HTTP tells us **how the browser and server communicate.**

TLS makes that communication **secure.**

Without TLS — HTTP

Imagine you log in:

Browser

|

| POST /login

| email=gopi@example.com

| password=hello123

↓

Server



With plain HTTP, the communication isn't protected against someone who can observe the network traffic.

Conceptually, an attacker observing the connection could see:

```
email=gopi@example.com  
password=hello123
```



That's obviously bad. 😬

With TLS — HTTPS

Now:

```
Browser  
|  
| encrypted HTTPS communication  
|  
↓  
Server
```



The data traveling over the network is encrypted.

Conceptually, instead of seeing:

```
password=hello123
```



someone intercepting the traffic sees something like:

```
8fA9x$2kLm...encrypted-looking-data...
```



They can't simply read the HTTP contents.

So what does TLS actually give us?

TLS provides three major protections:

1. 🗝️ Confidentiality

Other people shouldn't be able to read the data traveling between browser and server.

For example:

```
password  
session cookie
```



credit card information
personal data

are protected while in transit.

2. 🛡️ Integrity

TLS helps ensure that the data wasn't modified while traveling.

For example:

Browser → "Transfer ₹100"



An attacker shouldn't be able to modify it into:

Browser → "Transfer ₹10,000"



without the tampering being detected.

3. 🏠 Authentication

TLS certificates help the browser verify that it is communicating with the intended website.

For example:

https://bank.com



The browser checks the site's certificate and the certificate chain.

That's why you see the 🔒 in the browser.

But where does encryption actually happen?

This is the useful mental model:

Browser

|

|

↓

TLS encryption

=====

Internet



```
=====
|
| TLS decryption
↓
Server
```

The browser encrypts the HTTP data before sending it.

The server decrypts it after receiving it.

So technically:

```
HTTP request
  ↓
TLS encrypts it
  ↓
Internet
  ↓
TLS decrypts it
  ↓
Server receives HTTP request
```



That's why we say:

HTTPS is HTTP transported through a TLS-secured connection.

And this connects directly to cookies 🙄

Remember our authentication cookie:

```
Set-Cookie: sessionId=ABC123;
HttpOnly;
Secure;
SameSite=Lax
```

The **Secure** attribute means:

Send this cookie only over a secure connection, i.e. HTTPS.

So imagine:

```
Browser
|
| HTTPS
| Cookie: sessionId=ABC123
```



↓
Next.js Server

The session cookie is protected by TLS **while traveling over the network**.

But there's an important distinction:

HttpOnly and **TLS** solve different problems.

Protection	What it protects against
TLS / HTTPS	Someone intercepting network traffic
HttpOnly	JavaScript accessing the cookie
Secure	Preventing the cookie from being sent over HTTP
SameSite	Helps control cross-site cookie sending / CSRF risk

So:

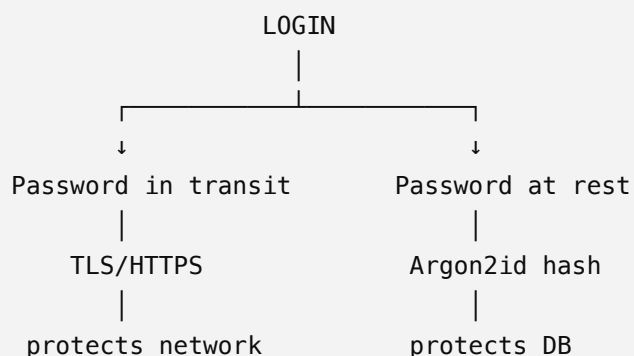
TLS protects the connection.

HttpOnly protects the cookie from JavaScript access.

One very important distinction

TLS doesn't encrypt your password in the database.

There are two completely different things happening:



For example:

Browser
|
| HTTPS/TLS
| password

```
↓  
Next.js Server  
  |  
  | Argon2id  
  |  
  ↓  
Database  
  |  
  | passwordHash  
  |  
  ↓  
$argon2id$v=19$...
```

So **TLS ≠ password hashing**.

TLS protects data **while traveling**.

Argon2id protects passwords **when stored**.

Interview-ready answer 🎯

"TLS stands for Transport Layer Security. It secures HTTP communication by providing encryption, integrity, and server authentication. HTTPS is essentially HTTP over a TLS-secured connection. For example, during login, TLS protects the password and session cookie while they're transmitted between the browser and server."

That distinction — **TLS protects data in transit**, while hashing protects passwords at **rest** — is a really good one to remember. 🔥



okay next

Absolutely 😊 Let's continue **Lesson 7 — Authentication & Authorization**.

We've covered **7.4 Login Flow** and the foundations of **7.5 Server Components + Auth**.

7.6 — Client Components + Authentication

This is where things get interesting because we now have **browser-side JavaScript** involved.

The key question is:

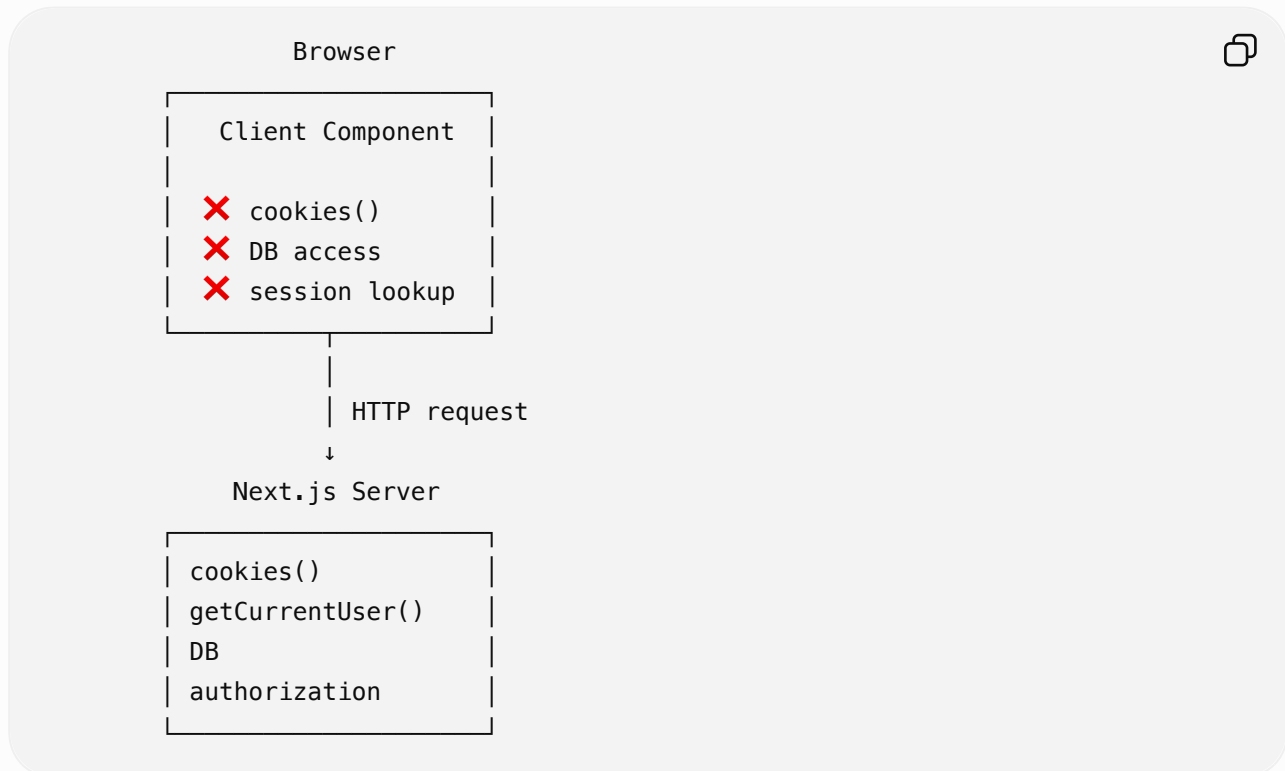
If the Client Component runs in the browser, how does it know whether the user is logged in?

And even more importantly:

Can a Client Component directly read the session cookie or query the database?

Short answer

No — not directly.



Why?

Because:

```
</> TypeScript
```

```
"use client";
```

means this component's JavaScript executes in the **browser**.

Whereas:

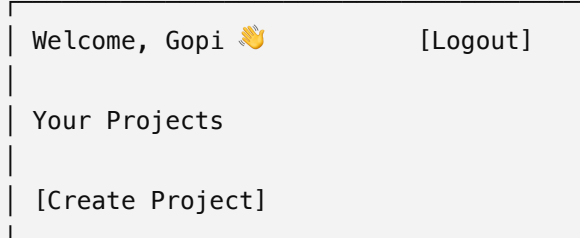
```
</> TypeScript
```

```
cookies()
db.user.findUnique(...)
process.env.SECRET
```

are server-side capabilities.

But the UI still needs to know the user

Imagine this dashboard:



```

Welcome, Gopi 🙌 [Logout]

Your Projects

[Create Project]
```

The **Logout button** might be a Client Component because it has interaction:

```
</> TypeScript

"use client";

export function LogoutButton() {
  return (
    <button onClick={...}>
      Logout
    </button>
  );
}
```

But the client shouldn't be doing:

```
</> TypeScript

// ❌ Browser
const user = await db.user.findUnique(...)
```

or:

```
</> TypeScript

// ❌ Browser
const sessionId = cookies().get("sessionId")
```

So how does it get the information?

There are **three common patterns**.

Pattern 1 — Server Component passes user to Client Component

This is often the simplest approach in Next.js.

Server Component

`</>` TypeScript

```
import { getCurrentUser } from "@lib/auth";
import { UserMenu } from "../UserMenu";

export default async function Dashboard() {
  const user = await getCurrentUser();

  if (!user) {
    redirect("/login");
  }

  return (
    <div>
      <h1>Welcome {user.name}</h1>

      <UserMenu
        user={{
          id: user.id,
          name: user.name,
          email: user.email,
        }}
      />
    </div>
  );
}
```

Dashboard is a **Server Component**.

Therefore:

Browser request

↓

Next.js Server

↓

cookies()

↓

session lookup

↓

user

↓

Dashboard Server Component

Then it passes safe data to the Client Component:

```
</> TypeScript

"use client";

export function UserMenu({ user }) {
  return (
    <div>
      <span>{user.name}</span>
      <button>Logout</button>
    </div>
  );
}
```

The boundary looks like:

SERVER

getCurrentUser()

↓

user data

↓

|

|

↓

serialized props

CLIENT

UserMenu

↓

interactive UI

Important ⚠

Don't do this:

```
</> TypeScript

<UserMenu user={user} />
```

if `user` contains sensitive fields such as:

```
passwordHash
resetToken
```

internalNotes
API keys

Instead:

</> TypeScript

```
<UserMenu
  user={{
    id: user.id,
    name: user.name,
    email: user.email,
  }}
/>
```

Only send what the client actually needs.

Pattern 2 — Client Component calls a Route Handler

Sometimes the Client Component needs to fetch the current user itself.

For example:

```
Client Component
  |
  | GET /api/me
  ↓
Next.js Route Handler
  |
  | reads session cookie
  ↓
Database
  |
  ↓
user
  |
  ↓
JSON response
  |
  ↓
Client Component
```

Route Handler:

</> TypeScript

```
// app/api/me/route.ts
```

```
import { getCurrentUser } from "@lib/auth";

export async function GET() {
  const user = await getCurrentUser();

  if (!user) {
    return Response.json(
      { user: null },
      { status: 401 }
    );
  }

  return Response.json({
    user: {
      id: user.id,
      name: user.name,
      email: user.email,
    },
  });
}
```

Then the Client Component:

```
</> TypeScript

"use client";

import { useEffect, useState } from "react";

export function Profile() {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch("/api/me")
      .then(res => res.json())
      .then(data => {
        setUser(data.user);
      });
  }, []);

  // ...
}
```

Notice something very important:

The Client Component **doesn't read the session cookie itself.**

It simply does:

```
GET /api/me
```

The browser automatically attaches the applicable cookie to the request.

```
</> http

GET /api/me
Cookie: sessionId=ABC123
```



Then the **server** reads the cookie.

Pattern 3 — Authentication library

In a real application, you might use an authentication library/provider that gives you APIs/hooks for accessing session state from both server and client.

Conceptually:

```
</> TypeScript

"use client";

const session = useSession();
```



Then:

```
</> TypeScript

if (session) {
  // show logged-in UI
}
```



But here's the BIG rule:

Client-side session state is for UI. Server-side authentication and authorization are the security boundary.

For example, hiding this button:

```
</> TypeScript

{user?.role === "ADMIN" && (
  <button>Delete User</button>
)}
```



does **NOT** secure the operation.

A malicious user can potentially call your server endpoint directly.

Therefore the server must still do:

`</> TypeScript`

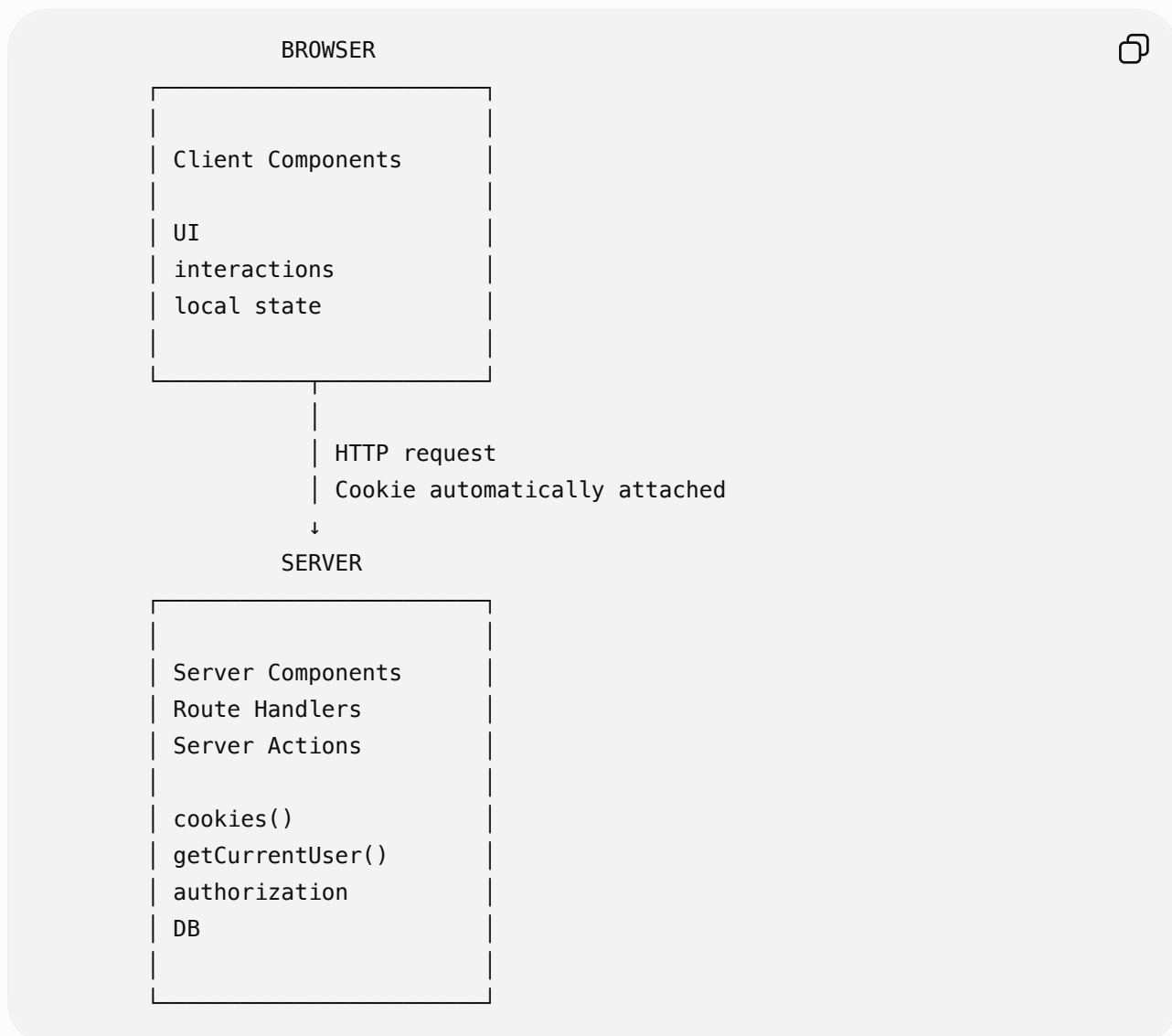
```
const user = await getCurrentUser();

if (!user) {
  return new Response("Unauthorized", { status: 401 });
}

if (user.role !== "ADMIN") {
  return new Response("Forbidden", { status: 403 });
}
```

The mental model I want you to remember 🧠

Think of it like this:



And the golden rule:

The client can display authentication state, but the server must enforce authentication and authorization.

That's the heart of **Client Components + Auth**.

One interview question you should be able to answer

Q: Can a Client Component directly access `cookies()` and query the database to determine the current user?

A strong answer:

"No. A Client Component executes in the browser, while `cookies()` from `next/headers` and database access are server-side capabilities. I would either read the authenticated user in a Server Component and pass safe data to the Client Component, or have the Client Component call a server endpoint such as `/api/me`. Regardless of how the UI gets the session state, authorization must always be enforced on the server."

Next, we should tackle **7.7 — Protecting Routes and Server Actions**, where we'll connect all of this to `401` vs `403`, route protection, authorization checks, and the common mistake of relying on client-side protection. 🔥





Awesome 😊 Let's move to the **next important Auth.js concept: how the session is actually used to protect pages and APIs.**

Lesson 7.x — Protecting Routes with Auth.js

We've now understood:

```
Google
↓
OAuth/OIDC
↓
Auth.js callback
↓
```



User/Account



Session



Cookie

Now the natural question is:

Once the user is logged in, how do I actually stop an unauthenticated user from accessing `/dashboard` or `/api/orders` ?

This is where `auth()` becomes really important.

1. Protecting a Server Component

Suppose we have:

```
app/  
├─ login/  
│   └─ page.tsx  
└─ dashboard/  
    └─ page.tsx
```



Our dashboard should only be accessible to logged-in users.

We can write:

`</>` TypeScript



```
// app/dashboard/page.tsx  
  
import { auth } from "@auth";  
import { redirect } from "next/navigation";  
  
export default async function DashboardPage() {  
  const session = await auth();  
  
  if (!session) {  
    redirect("/login");  
  }  
  
  return (  
    <div>  
      <h1>Welcome {session.user?.name}</h1>  
      <p>{session.user?.email}</p>  
    </div>  
  );  
}
```

```
);  
}
```

Now let's understand **exactly what happens**.

2. User requests `/dashboard`

Browser:

```
</> http  
  
GET /dashboard  
Cookie: authjs.session-token=ABC123
```



Next.js receives this request.

Because `dashboard/page.tsx` is a **Server Component**, it executes on the server.

Then:

```
</> TypeScript  
  
const session = await auth();
```



runs.

3. What does `auth()` do?

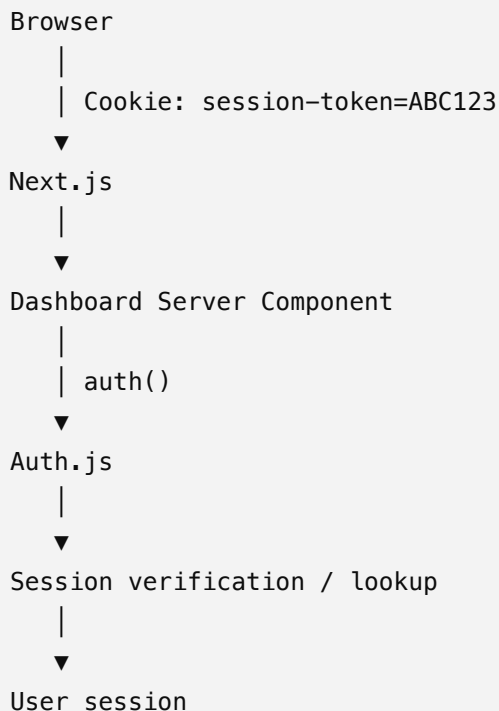
Remember where it came from:

```
</> TypeScript  
  
// auth.ts  
  
export const {  
  handlers,  
  auth,  
  signIn,  
  signOut  
} = NextAuth({...});
```



So `auth()` is an Auth.js server-side function.

Conceptually:



If the session is valid:

</> TypeScript

```
session = {
  user: {
    name: "Gopi",
    email: "gopi@example.com"
  }
}
```

Then the page renders.

4. What if the user isn't logged in?

Suppose there is no valid session.

Then:

</> TypeScript

```
const session = await auth();
```

returns something equivalent to:

</> TypeScript

```
null
```

So:

```
</> TypeScript
```

```
if (!session) {  
  redirect("/login");  
}
```



executes.

Next.js sends a redirect response.

Conceptually:

```
</> http
```

```
307 Temporary Redirect  
Location: /login
```



The browser then requests:

```
</> http
```

```
GET /login
```



So:

```
User → /dashboard
```

```
  ↓  
  auth()  
  ↓  
no session  
  ↓  
redirect("/login")  
  ↓  
  /login
```



5. Why isn't this authorization?

This is an important distinction.

This code checks:

```
</> TypeScript
```



```
if (!session)
```

That is **authentication**.

We're asking:

"Is this person logged in?"

But suppose we have:

Admin Dashboard



and only admins can access it.

Then we need **authorization**.

For example:

```
</> TypeScript
```



```
const session = await auth();

if (!session) {
  redirect("/login");
}

if (session.user.role !== "ADMIN") {
  redirect("/forbidden");
}
```

Now we're asking two different questions:

Authentication:

"Who are you / are you logged in?"



Authorization:

"Are you allowed to do this?"

6. But there's an important security rule

Never rely only on hiding UI.

For example:

```
</> TypeScript
```



```
{session?.user.role === "ADMIN" && (  
  <button>Delete User</button>  
)}
```

This is useful for UX.

But it is **not security**.

A malicious user can manually send a request to your server.

For example:

```
</> http  
  
POST /api/users/delete
```



Therefore your server-side action/API must check authorization too.

7. Protecting a Route Handler

Suppose we have:

```
app/api/orders/route.ts
```



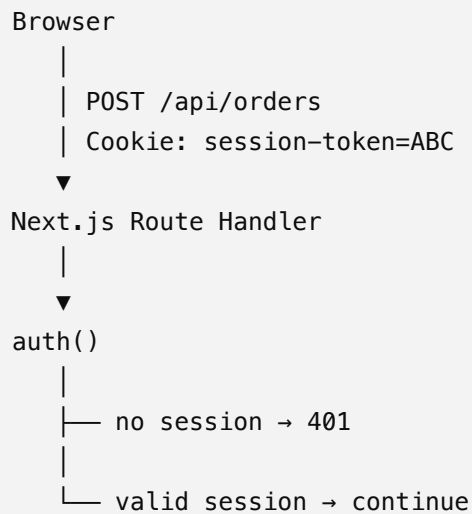
We can do:

```
</> TypeScript  
  
import { auth } from "@auth";  
import { NextResponse } from "next/server";  
  
export async function POST(request: Request) {  
  const session = await auth();  
  
  if (!session) {  
    return NextResponse.json(  
      { error: "Unauthorized" },  
      { status: 401 }  
    );  
  }  
  
  // User is authenticated  
  // Continue processing order...  
  
  return NextResponse.json({  
    success: true,  
  });  
}
```



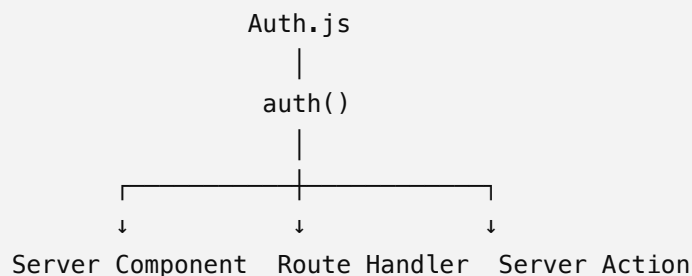
```
});  
}
```

Now:



8. This is why `auth()` is so useful

You can use the same server-side authentication context in multiple places:



For example:

Server Component

</> TypeScript

```
const session = await auth();
```

Route Handler

</> TypeScript

```
const session = await auth();
```

Server Action

</> TypeScript



```
const session = await auth();
```

The exact mechanics differ depending on the execution context, but the idea is:

Your server-side code asks Auth.js to resolve the current authenticated user/session.

9. Server Action example

Suppose:

</> TypeScript



```
"use server";

import { auth } from "@auth";

export async function deleteAccount() {
  const session = await auth();

  if (!session) {
    throw new Error("Unauthorized");
  }

  // Delete account...
}
```

Even if your UI hides the "Delete Account" button from logged-out users, the Server Action still checks authentication.

That's the security boundary.

10. Interview answer 🎯

If an interviewer asks:

"How would you protect a page in Next.js using Auth.js?"

A strong answer would be:

"For server-rendered pages, I can call Auth.js's server-side `auth()` function to resolve the current session. If there's no valid session, I redirect the user to the login page. For protected mutations or APIs, I perform the authentication and authorization checks again inside the Server Action or Route Handler rather than relying on client-side UI checks."

And if they ask:

"Is hiding an admin button authorization?"

Say:

"No. That's only a UI concern. Authorization must be enforced on the server at the resource or mutation boundary."

One thing I want you to notice

We now have **three different places** where Auth.js can participate:

1. LOGIN

```
signIn()  
  ↓  
OAuth  
  ↓  
callback  
  ↓  
session created
```



2. READING SESSION

```
auth()  
  ↓  
"Who is currently logged in?"
```

3. PROTECTING ACTIONS

```
auth()  
  ↓  
authenticate  
  ↓  
authorize  
  ↓  
perform operation
```

That distinction is **very interview-important**.

Next after this

The natural next piece is **Auth.js callbacks** — specifically:

`</> TypeScript`



```
callbacks: {
  async jwt() {},
  async session() {},
  async signIn() {},
  async redirect() {}
}
```

Because you've now seen `session` being created and consumed, the next question is:

"How do I customize what gets stored in the JWT/session — for example, how do I put a user's `role` or `id` into the session?"

That's where the `jwt()` and `session()` callbacks finally make sense. 🔥



how many more are there

Not too many 😊 We're actually pretty close to wrapping the **Authentication & Authorization** section.

Based on what we've covered so far, I'd structure the remaining part like this:

Remaining Auth.js / Auth & Authorization lessons

#	Topic	What you'll learn
7.6	Route Protection	<code>auth()</code> , protecting pages, APIs, Server Actions
7.7	Auth.js Callbacks	<code>signIn</code> , <code>jwt</code> , <code>session</code> , <code>redirect</code> callbacks
7.8	Session Customization	Put <code>user.id</code> , <code>role</code> , etc. into the session
7.9	Authorization / RBAC	Roles, permissions, admin/user access
7.10	Session Strategies	Database sessions vs JWT sessions

#	Topic	What you'll learn
7.11	 CSRF + SameSite	Why state-changing requests need protection
7.12	 Middleware / Proxy Auth	Protecting groups of routes before rendering
7.13	 Logout & Session Expiry	Sign-out, expiration, invalid/expired sessions
7.14	 Complete Auth Architecture	Put Browser → Next.js → Auth.js → OAuth → DB → Session together
7.15	 Auth Interview Questions	Real interview scenarios + architecture questions

So **about 10 lessons remain**, depending on how deeply we split things.

But some of these are short. The **big ones** are:

1. Callbacks
2. Session customization
3. Authorization/RBAC
4. JWT vs DB sessions
5. CSRF/SameSite
6. Middleware/Proxy

After that, I'd consider the Auth section **complete** and move you to the next major Next.js/interview area. 🚀

And I wouldn't rush through these—especially callbacks and authorization. Given the way you've been learning, I'd rather make sure you can explain the **actual request flow** than just memorize Auth.js APIs.



so after this authentication and authorization are done?

Yes. 👍 After we finish those remaining Auth.js topics, we can consider the **Authentication & Authorization section complete**.

More precisely, your **Next.js Auth & Authorization roadmap** will cover:

AUTHENTICATION



- ✓ Authentication vs Authorization
- ✓ Auth architecture
- ✓ Cookies
- ✓ Sessions
- ✓ JWT
- ✓ Password hashing
- ✓ Login flow
- ✓ Server Components + auth
- ✓ Client Components + auth
- ✓ Auth.js
- ✓ OAuth 2.0
- ✓ OpenID Connect
- ✓ Google authentication
- ✓ Authorization code + callback
- ✓ Client ID / Client Secret
- ✓ Token endpoint
- ✓ Auth.js adapters
- ✓ ORM + database persistence
- ✓ User / Account / Session
- ✓ Database sessions vs JWT sessions
- ✓ Route protection
- ✓ Auth.js callbacks
- ✓ Session customization

AUTHORIZATION

- ✓ Authentication vs authorization
- ✓ Protecting Server Components
- ✓ Protecting Route Handlers
- ✓ Protecting Server Actions
- ✓ Roles / RBAC
- ✓ Permissions
- ✓ Server-side authorization
- ✓ Client UI vs actual security boundary

SECURITY

- ✓ HttpOnly / Secure / SameSite
- ✓ CSRF
- ✓ Session expiry
- ✓ Logout
- ✓ Middleware / Proxy protection
- ✓ Common auth vulnerabilities

Then we'll do **one complete end-to-end architecture walkthrough**:



```
graph TD
  Browser --> Login
  Login --> Authjs[Auth.js]
  Authjs --> Google[Google OAuth/OIDC]
  Google --> Callback
  Callback --> Exchange[Google token exchange]
  Exchange --> User[User + Account]
  User --> Adapter
  Adapter --> Database
  Database --> Session
  Session --> Cookie
  Cookie --> Request[Protected request]
  Request --> auth[auth()]
  auth --> Authentication
  Authentication --> Authorization
  Authorization --> Resource[Protected resource]
```

That final walkthrough is important because you've been asking the right kind of **"but who actually calls what?"** questions. 😊 It will tie all the individual pieces together.

So yes: once we finish these, you can confidently mark *Next.js Authentication & Authorization* as DONE. 🗝️✅



ChatGPT can make mistakes. Check important info.